

UNIT II - EXPRESS JS

Express JS and features–Routing – HTTP request and response–Middleware–Error Handling.

Overview

Express JS is a web application framework that provides you with a simple API to build websites, web apps and back ends. Express provides a minimal interface to build our applications. It provides us the tools that are required to build our app. It is flexible as there are numerous modules available on **npm**, which can be directly plugged into Express.

What is EXPRESS JS?

ExpressJS is a minimal and flexible web application framework for Node.js. It provides a robust set of features to develop web and mobile applications. ExpressJS simplifies the process of building server-side web applications and APIs by offering a lightweight structure with a variety of HTTP utility methods and middleware.

INTRODUCTION

- ExpressJS is a bridge between the front-end and back-end of a web application.
- It acts as the intermediary that processes data and information between the two, making it possible for users to interact with the application in real-time.
- This framework also includes a range of helpful features, such as middleware, routing, and templating that make it easier to develop and maintain a web application.
- ExpressJS is a free and open-source web application framework that is built on top of Node.js.
- It's designed to simplify the process of building and maintaining server-side applications. With ExpressJS, you can create dynamic and interactive websites, build RESTful APIs, and handle HTTP requests and responses in an organized and efficient manner.
- ExpressJS is a great choice for beginners and experienced web developers alike. Its simple yet powerful architecture makes it easy to learn and use, while its robust set of features makes it suitable for creating complex web applications

What is Express JS used for?

ExpressJS is a versatile web application framework that is used for a variety of purposes.

1. Building dynamic websites – ExpressJS can be used to create interactive and dynamic websites that allow users to input and retrieve data in real time.

2. Developing RESTful APIs – ExpressJS makes it easy to build RESTful APIs, which are a type of web service that allows different applications to communicate with one another.

Feature	ExpressJS	NodeJS
Purpose	Web framework	JavaScript runtime
Built on	NodeJS	Chrome's V8 JavaScript engine
Focus	Server-side development	General-purpose JavaScript programming
Key features	Routing, middleware, templating	Non-blocking I/O, event-driven programming
Use Cases	Building web applications, RESTful APIs	Server-side scripting, creating command line tools

EXPRESS JS ADVANTAGES

ExpressJS is a popular web application framework that offers several advantages to developers building web applications. Below are some of the key advantages of ExpressJS:

Simplicity

ExpressJS has a simple and intuitive API that makes it easy to learn and use, even for beginner web developers. This makes it a great choice for building small to medium-sized web applications.

Performance

ExpressJS is designed to be fast and efficient, with a lightweight architecture that allows it to handle large amounts of traffic with ease.

Flexibility

ExpressJS is highly flexible and can be used to build a wide range of web applications, from simple websites to complex APIs. This makes it a versatile choice for web developers.

Integrations

ExpressJS can be easily integrated with other popular technologies and frameworks, such as databases, templating engines, and more. This makes it simple to build complex and feature-rich web applications.

EXPRESS JS ROUTING

Routing refers to how an application's endpoints (URIs) respond to client requests. It determines the way requests are handled and responses are sent in a web application. ExpressJS provides a simple and powerful way to handle routing.

- Routing in ExpressJS is a powerful feature that provides a structured way to handle various HTTP requests.
- It allows defining clear endpoints, handling dynamic URL segments through route parameters, utilizing query parameters for additional request data, and employing middleware to execute code during the request-response cycle.
- The ability to group routes and combine multiple handlers further enhances the flexibility and maintainability of Express applications.
- Routing in ExpressJS is used to subdivide and organize the web application into multiple mini-applications each having its own functionality.
- It provides more functionality by subdividing the web application rather than including all of the functionality on a single page.
- These mini-applications combine together to form a web application. Each route in Express responds to a client request to a particular route/endpoint and an HTTP request method (GET, POST, PUT, DELETE, UPDATE and so on).
- Each route basically refers to the different URLs in the website. So when a URL (Eg: www.abcd.org/login) matches a route then the function associated with that specific route is executed.

Route Methods:

- These methods correspond to HTTP methods and define the response for specific HTTP requests.
- Common methods include `app.get()`, `app.post()`, `app.put()`, and `app.delete()`.

```
const express = require('express');
const app = express();
app.get('/', (req, res) => {
  res.send('Hello, World!');
});
app.post('/submit', (req, res) => {
  res.send('Form submitted!');
});
app.put('/update', (req, res) => {
  res.send('Data updated!');
});
app.delete('/delete', (req, res) => {
```

```
    res.send('Data deleted!');
  });
  app.listen(3000, () => {
    console.log('Server running on http://localhost:3000');
  });
```

Route Paths:

- A route path is used to match the request URL.
- Paths can be strings, string patterns, or regular expressions.

```
app.get('/about', (req, res) => {
  res.send('About Page');
});
```

Route Handlers:

- Route handlers are functions executed when the route is matched.
- They can be single callback functions or arrays of functions.

```
app.get('/contact', (req, res) => {
  res.send('Contact Page');
});
```

Route Parameters:

Route parameters are named URL segments used to capture values at specific positions in the URL. They are defined with a colon (:) prefix.

```
app.get('/user/:userId', (req, res) => {
  res.send(`User ID: ${req.params.userId}`);
});
```

Query Parameters:

Query parameters are optional parts of a URL that come after the symbol. They are typically used to filter or sort resources.

```
app.get('/search', (req, res) => {
  const query = req.query.query;
  res.send(`Search query: ${query}`);
});
```

Middleware in Routing

Middleware functions execute during the request-response cycle and have access to the request object (`req`), the response object (`res`), and the next middleware function in the cycle.

```
const logger = (req, res, next) => {  
  console.log(`${req.method} request for '${req.url}'`);  
  next();  
};  
app.use(logger);  
app.get('/', (req, res) => {  
  res.send('Home Page');  
});
```

Route Groups

Route groups allow you to apply middleware to multiple routes using a router instance.

```
const router = express.Router();  
router.use((req, res, next) => {  
  console.log('Middleware for all routes in this group');  
  next();  
});  
router.get('/profile', (req, res) => {  
  res.send('Profile Page');  
});  
router.get('/settings', (req, res) => {  
  res.send('Settings Page');  
});  
app.use('/user', router);
```

Routers

Routers are very helpful in separating concerns and keep relevant portions of our code together. They help in building maintainable code.

Routers in ExpressJS are used to create modular, mountable route handlers. A router instance is a complete middleware and routing system, which you can attach to the main application.

Routers help in organizing and managing different parts of the application effectively by grouping the routes.

Key Concepts of Routers

1. Router Instance:
 - A mini-application capable of performing middleware and routing functions.
2. Creating a Router:
 - A router is created using `express.Router()`.
 - Router instances can be treated as isolated middleware and routing systems.
3. Mounting Routers:

Routers can be mounted on the main application or other routers to create nested route structures.

Creating and Using Routers

Basic Example of Creating a Router:

1. Create a Router:

```
const express = require('express');
const app = express();
const router = express.Router();
// Define routes in the router
router.get('/users', (req, res) => {
  res.send('List of users');
});
router.post('/users', (req, res) => {
  res.send('Create a new user');
});
// Use the router in the application
app.use('/api', router);
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```
2. Organizing Routes:
 - Separate route definitions into different files for better organization.

```
// routes/users.js
const express = require('express');
const router = express.Router();
router.get('/', (req, res) => {
  res.send('List of users');
```

```

});
router.post('/', (req, res) => {
  res.send('Create a new user');
});
module.exports = router;
// app.js
const express = require('express');
const app = express();
const usersRouter = require('./routes/users');
app.use('/api/users', usersRouter);
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});

```

EXPRESS.JS REQUEST AND RESPONSE

- In ExpressJS, handling client requests and sending responses is fundamental.
- The `req` (request) and `res` (response) objects provide crucial information and functionality to manage the HTTP request-response lifecycle.
- The request object represents the HTTP request and contains properties for the request query string, parameters, body, HTTP headers, etc.

Syntax: `app.get('/', function (req, res) { })`

1. Request Object (req)

The request object represents the HTTP request and contains properties for the request query string, parameters, body, HTTP headers, and more.

Key Properties and Methods of req:

`req.params`: Contains route parameters.

```

app.get('/user/:userId', (req, res) => { res.send(`User ID:
${req.params.userId}`);
});

```

1. `req.query`: Contains query string parameters.

```

app.get('/search', (req, res) => { const query
= req.query.q; res.send(`Search query:
${query}`);

```

```
});
```

2. req.body: Contains the body data of the request (requires body-parsing middleware like body-parser).

```
const bodyParser = require('body-parser');
app.use(bodyParser.json());
app.post('/login', (req, res) => {
  const username = req.body.username;
  const password = req.body.password;
  res.send(`Username: ${username}, Password: ${password}`);
});
```

- 3.req.headers: Contains the request headers.

```
app.get('/headers', (req, res) => {
  res.send(req.headers);
});
```

- 4.req.method: The HTTP method of the request.

```
app.use((req, res, next) => { console.log(`Request
Method: ${req.method}`); next();
});
```

- 5.req.url: The URL of the request.

```
app.use((req, res, next) => {
  console.log(`Request URL: ${req.url}`);
  next();});
```

Request Object Properties

S.No	Properties	Description
1	.app	used to hold a reference to the instance of the express application.
2	.body	Contains key-value pairs of data submitted in the request body.
3	.cookies	is property contains cookies sent by the request, used for the
		cookie-parser middleware.

4	.ip	.ip is remote IP address of the request
5	.path	.path contains the path part of the request URL.
6	.route	.route is currently-matched route.

2. Response Object (res)

The response object represents the HTTP response that an Express application sends when it receives an HTTP request. It contains methods for sending responses back to the client.

Key Methods of res:

res.send(): Sends a response of various types (string, JSON, buffer). `app.get('/', (req, res) => { res.send('Hello, World!'); });`

res.json(): Sends a JSON response.

```
app.get('/user', (req, res) => {
    res.json({ name: 'John Doe', age: 30 });
});
```

res.status(): Sets the HTTP status code. `app.get('/not-found', (req, res) => { res.status(404).send('Not Found'); });`

res.sendFile(): Sends a file as an octet stream. `const path = require('path');`

```
app.get('/file', (req, res) => {
    res.sendFile(path.join(__dirname, 'file.txt'));
});
```

res.redirect(): Redirects the request to a different URL.

```
app.get('/redirect', (req, res) => {
    res.redirect('https://www.example.com');
});
```

res.render(): Renders a view template. `app.set('view engine', 'pug');`

```
app.get('/template', (req, res) => {res.render('index', { title: 'Express', message: 'Hello there!' });});
```

Response Object

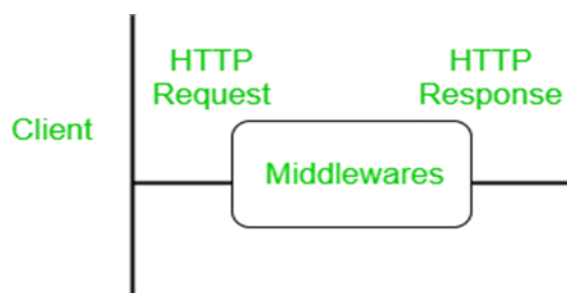
The response object specifies the HTTP response when an Express app gets an HTTP request. The response is sent back to the client browser and allows you to set new cookies value that will write to the client browser.

S. No	Properties	Description
1	res.app	res.app is hold a reference to the instance of the express application that is using the middleware.
2	res.locals	Specify an object that contains response local variables scoped to the request.

Using the `req` and `res` objects in ExpressJS, developers can effectively handle client requests and send appropriate responses, enabling the development of dynamic and responsive web applications.

MIDDLEWARE IN EXPRESS JS

- ✓ Express.js is a routing and Middleware framework for handling the different routing of the webpage and it works between the request and response cycle.
- ✓ Middleware gets executed after the server receives the request and before the controller actions send the response.
- ✓ Middleware has the access to the request object, responses object, and next, it can process the request before the server send a response.
- ✓ An Express-based application is a series of middleware function calls.



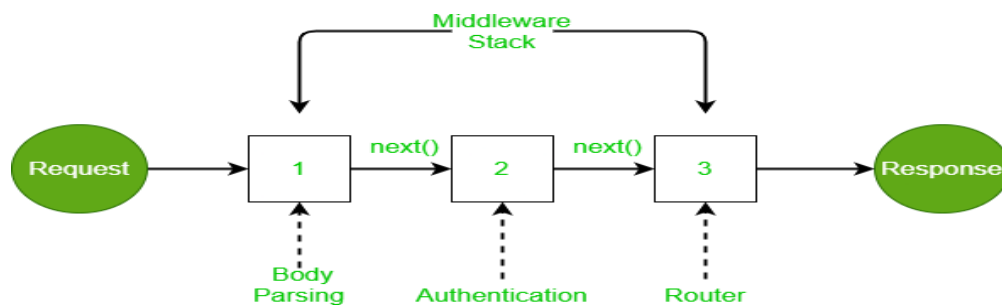
Advantages of using middleware:

- Middleware can process request objects multiple times before the server works for that request.
- Middleware can be used to add logging and authentication functionality.
- Middleware improves client-side rendering performance.
- Middleware is used for setting some specific HTTP headers.

- Middleware helps for Optimization and better performance.

Middleware Chaining: Middleware can be chained from one to another, hence creating a chain of functions that are executed in order. The last function sends the response back to the browser. So, before sending the response back to the browser the different middleware processes the request. The `next()` function in the express is responsible for calling the next middleware function if there is one.

Modified requests will be available to each middleware via the next function –



In the above case, the incoming request is modified and various operations are performed using several middlewares, and middleware is chained using the next function. The router sends the response back to the browser.

Middleware Syntax: The basic syntax for the middleware functions are as follows –

```
app.get(path, (req, res, next) => {}, (req, res) => {})
```

Middleware functions take 3 arguments: the **request object**, the **response object**, and the **next function** in the application's request-response cycle, i.e., two objects and one function.

TYPES OF MIDDLEWARES:

- **Application-level Middleware:** Bound to an instance of `express()`.
- **Router-level Middleware:** Bound to an instance of `express.Router()`.
- **Built-in Middleware:** Provided by Express (e.g., `express.static`, `express.json`, `express.urlencoded`).
- **Error-handling Middleware:** Defined with four arguments (`err`, `req`, `res`, `next`).

Application-Level Middleware

Purpose: These middleware functions are bound to an instance of the Express application using `app.use()` OR `app.METHOD()`.

Example: **Logging Middleware:** Logs details about each request coming to the application.

```
const express = require('express');
const app = express();
const logger = (req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next(); // Pass control to the next middleware or route handler
};
app.use(logger);
app.get('/', (req, res) => {
  res.send('Home Page');
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

1. Router-Level Middleware

Purpose: These middleware functions are bound to an instance of `express.Router()` and can be used for specific route paths.

Example: Authentication Middleware: Checks if the user is authenticated before allowing access to certain routes.

```
const express = require('express');
const app = express();
const router = express.Router();
const auth = (req, res, next) => {
  if (req.isAuthenticated()) {
    next(); // Pass control to the next middleware or route handler
  } else {
    res.status(401).send('Unauthorized');
  }
};

router.use(auth);
router.get('/profile', (req, res) => {
  res.send('User Profile');
});
```

```
app.use('/user', router);
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

2. Error-Handling Middleware

Purpose: These middleware functions catch and process errors, providing a centralized way to handle errors.

Example: Error Logger: Logs errors and sends a response to the client.

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  throw new Error('Something went wrong!');
});

app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send('Something broke!');
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

3. Built-in Middleware

Purpose: These middleware functions come with ExpressJS and provide common functionalities.

Examples:

- **express.json():** Parses incoming requests with JSON payloads.
- **express.static():** Serves static files such as images, CSS files, and JavaScript files.

```
const express = require('express');
```

```
const app = express();
// Parse JSON payloads
app.use(express.json());
```

```
// Serve static files
app.use(express.static('public'));
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

ERROR HANDLING IN EXPRESS

- ❖ Error handling in Express is referred to as something that handles or processes errors that may come while executing any synchronous code or asynchronous code.
- ❖ Error handling in ExpressJS ensures that your application can gracefully manage and respond to errors that occur during its execution.
- ❖ This process involves defining how your application reacts to various types of errors, from user mistakes to unexpected issues in your code.
- ❖ Error handling middleware functions are similar to other middleware functions but have an additional argument: **err**.
- ❖ This argument captures the error object, allowing the middleware to handle it appropriately.
- ❖ Why is Error Handling Important?
 1. Enhances user experience by providing informative error messages.
 2. Prevents application crashes by managing unexpected issues gracefully.
 3. Helps in debugging and maintaining code by logging error details.
- ❖ Example:- A basic example of an error handling middleware function:

```
p.get('/', (req, res) => {
  throw new Error('Something went wrong!');
};
p.use((err, req, res, next) =>
{ console.error(err.stack);
```

```
res.status(500).send('Something broke!');
;
```

Explanation:

- **Route Definition with an Error:** Throws an error when a GET request is made to the root URL.

- **Error Handling Middleware:** Logs the error and sends a 500 (Internal Server Error) status code and a message to the client.
 - ✓ `app.use((err, req, res, next) => { ... });` - This middleware function is specifically for handling errors. It has four parameters:
 - `err`: The error object that was thrown.
 - `req`: The request object.
 - `res`: The response object.
 - `next`: The next middleware function in the stack.
 - ✓ `console.error(err.stack);` - Logs the error stack trace to the console for debugging purposes.
 - ✓ `res.status(500).send('Something broke!');` - Sends a 500 (Internal Server Error) status code and a message to the client, indicating that an error occurred.

Handling Different Types of Errors

1. Example: Handling 404 and Other Errors

```
const express = require('express');
const app = express();
app.use(express.json());
app.get('/', (req, res) => {
  res.send('Home Page');
});
// Middleware to handle 404 errors
app.use((req, res, next) => {
  res.status(404).send('Sorry, we could not find that!');
});
// Error handling middleware
app.use((err, req, res, next) => {
  console.error(err.stack);

  res.status(500).send('Something broke!');
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

Ex
pla
nat
ion:

ends a 404-status code and a message when the requested resource is not found.

- **General Error Handling Middleware:** Logs the error and sends a 500-status code and message for other errors.

2. Custom Error Handling

• Creating custom error classes allows for more specific error handling.

0

4

E

r

r

o

r

H

a

n

d

l

i

n

g

M

i

d

d

l

e

w

a

r

e

:

S

Example: Custom Error Class and

Handling const express

```
=    require('express');
```

```
const app = express();
```

```
// Custom error class
```

```
class NotFoundError
```

```
  extends Error {
```

```
    constructor(message) {
```

```
      super(message);
```

```
      this.name = 'NotFoundError';
```

```
    }
```

```
  }
```

```
app.use(express.
```

```
  json());
```

```
  app.get('/',
```

```
(req, res) => {
```

```
  res.send('Hom
```

```
    e Page');
```

```
});
```

```
app.get('/error', (req, res, next) => {
```

```
  const error = new NotFoundError("This resource was not
```

```
  found'); next(error);
```

```
});
```

```
// Middleware to handle custom
```

```
NotFoundError app.use((err, req, res, next)
```

```
=> {
```

```
  if (err instanceof
```

```
    NotFoundError) {
```

```
      res.status(404).send(err.m
```

```
      essage);
```

```
    } else
```

```
    {  
      next  
      (err)  
    ;});  
  }  
});  
// General error handling middleware  
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).send('Something broke!');  
});  
app.listen(3000, () => {  
  console.log('Server running on http://localhost:3000');
```

Explanation:

- **Custom Error Class:** Defines a `NotFoundError` class for specific not found errors.
- **Throwing a Custom Error:** Uses the custom error class to throw a specific error.
- **Handling Custom Errors:** Checks if the error is an instance of `NotFoundError` and handles it separately.
- **General Error Handling:** Handles all other errors not specifically caught by the custom error handler.

Effective error handling in ExpressJS ensures that your application can manage unexpected issues gracefully, providing a better experience for users and easier debugging for developers. By implementing proper error handling middleware, differentiating error types, and using custom errors, you can create a robust and resilient application.